



Overview



REMEDIATION REPORT
for
MEGAQUAGGA PUBLISHING



MQP

Version 1.0 (Nov 22, 2025)

Prepared by S M Mohiuddin Mahmood

Presented to **MEGAQUAGGA PUBLISHING**

DISCLAIMER: This document contains proprietary and confidential information. 0x2A SECURITY guarantees the confidentiality of the information and any other material that has or will be disclosed during the performance of this service. A mutual non-disclosure agreement of confidential information is assumed between 0x2A SECURITY and MEGAQUAGGA PUBLISHING, limiting the use of confidential information exclusively for purposes associated with properly delivering this service.

EXECUTIVE SUMMARY

The primary goal of this remediation process was to significantly strengthen the security posture of the MegaQuagga infrastructure against known threats. Key security measures were implemented, including inspecting and sniffing network traffic to understand and trace exploit attempts, configuring a ModSecurity Web Application Firewall (WAF) to filter malicious requests, and implementing a proxy server to better control and monitor traffic flow. We also completed a full offloading process to enhance system performance and security, and then thoroughly verified and documented all changes to confirm their effectiveness.

These remediations were specifically designed to address critical vulnerabilities that could lead to unauthorized access, data loss, or service disruption. The implementation of the WAF and proxy server directly mitigates risks associated with common web application attacks, such as SQL injection and cross-site scripting (XSS), by blocking known malicious patterns before they can reach the server. By inspecting and sniffing network packets, we addressed the risk of unseen data exfiltration and covert channel communication. Collectively, these steps greatly reduce the overall security risk to MegaQuagga, ensuring the system is more resilient and trustworthy.



Actions Taken

ACTIONS TAKEN

This section outlines the actions taken on which assets to remediate which vulnerabilities.

Vulnerability	Remediation Solution
WordPress is vulnerable to web application attacks	Install ModSecurity Web Application Firewall (WAF)
Unencrypted web traffic	Implement the HTTPS Protocol on the reverse proxy.

Remediation Solution #1: Install ModSecurity WAF

Remediation: ModSecurity WAF Configuration

1. **Preparation & Rule Analysis:** Reviewed ModSecurity architecture and the **OWASP Core Rule Set (CRS)** to identify how generic attack signatures are matched.
2. **Passive Testing:** Deployed ModSecurity in "**Passive Detection**" mode and ran known exploits against the server to confirm detection without blocking traffic.
3. **Log Verification:** Monitored **Graylog** to verify that the specific exploit attempts generated the expected alerts.
4. **Active Remediation:** Edited the configuration file to switch ModSecurity from "Passive Detection" to "**Active Prevention**" mode (SecRuleEngine On).
5. **Final Validation:** Re-ran the exploits to confirm they were successfully blocked (resulting in **403 Forbidden** errors) and verified the corresponding "blocked" events in Graylog.

Remediation Solution #2: Implement the HTTPS Protocol

Remediation: Encrypting Web Traffic (HTTPS Implementation)

- **Asset:** Nginx Reverse Proxy Server
- **Vulnerability:** Clear-text transmission of data (Lack of Encryption). This was **discovered** in the previous sprint during network traffic analysis (packet capture), where sensitive user credentials were observed passing through the network in plain text.
- **Fix Type:** Configuration change (SSL/TLS Implementation).

Steps Taken:

1. **Certificate Generation:** Generated a self-signed SSL certificate and private key using **OpenSSL** on the proxy server to enable encrypted communication channels.
2. **Proxy Configuration:** Modified the Nginx configuration file (typically `/etc/nginx/sites-available/default`) to enable SSL.
3. **Port & Key Setup:** Configured the server block to **listen on port 443** (HTTPS) and defined the file paths for the newly created certificate and private key.
4. **Service Restart:** Restarted the Nginx service (`systemctl restart nginx`) to apply the new security configurations.
5. **Verification:** Validated the fix by accessing the web application via `https://` and using `curl` to confirm that traffic was now encrypted and no longer readable in plain text.

☒ Evidence of Remediation

EVIDENCE OF REMEDIATION

This section documents evidence of the remediations being implemented. This evidence may take the form of details from packet captures (for example, alterations in traffic flow or packet headers), ModSecurity logs appearing in the Graylog SIEM, or output from command-line tools. For each remediation, we provide evidence of the original state as well as the current altered state.

Reverse proxy

These are alterations to MegaQuagga’s environment associated with implementing the reverse proxy server responsible for implementing our two remediations (WAF & SSL offloading).

Item	Original Vulnerable State	Current Remediated State
/etc/nginx/sites-available/default	<pre>server { listen 80; ... }</pre> (Note: No SSL configuration present)	<pre>server { listen 443 ssl; ssl_certificate /etc/ssl/certs/nginx-selfsigned.crt; ssl_certificate_key /etc/ssl/private/nginx-selfsigned.key; }</pre>
Wireshark Capture (PCAP) (e.g., http_traffic.pcap vs https_traffic.pcap)	Packet # [PfSense-LAN-2.pcap]: Traffic is sent over HTTP. The protocol shows "HTTP". Payload (including passwords) is visible in cleartext in the detailed packet view.	Packet # [pfsense-lan-4.pcap]: Traffic is sent over TLSv1.2/TLSv1.3. The protocol shows "TLS". Payload is encrypted and unreadable in the detailed packet view.

Web Application Firewall (WAF)

These are alterations to MegaQuagga’s environment associated with implementing the WAF remediation.

Item	Original Vulnerable State	Current Remediated State
/etc/modsecurity/modsecurity.conf	SecRuleEngine DetectionOnly (WAF only logs events but does not block them)	SecRuleEngine On (WAF actively blocks malicious requests)
Graylog Alerts / Logs	Status: 200 OK Exploit attempts (e.g., SQL Injection) are allowed through to the server. Logs show successful request codes.	Status: 403 Forbidden Exploit attempts are blocked by ModSecurity. Logs show a 403 error code, indicating the WAF intercepted the attack.

SSL Offloading

These are alterations to MegaQuagga's environment associated with implementing the WAF remediation.

Item	Original Vulnerable State	Current Remediated State
Nginx Proxy Configuration (/etc/nginx/sites-available/default)	Listen 80; The server was configured to listen only on port 80 (HTTP). No SSL directives were present. (Traffic is unencrypted)	Listen 443 ssl; The server is configured to listen on port 443. SSL certificate and key paths are defined to offload encryption. (Traffic is encrypted)
pfSense Firewall Rule	Allow IPv4 TCP Port 80 Firewall rule permitted traffic on destination port 80 (HTTP) to the	Allow IPv4 TCP Port 443 Firewall rule modified/added to permit traffic on

Item	Original Vulnerable State	Current Remediated State
<i>(Firewall > Rules > WAN)</i>	WAN address.	destination port 443 (HTTPS) to the WAN address.
pfSense NAT Rule <i>(Firewall > NAT > Port Forward)</i>	Interface: WAN / Dest. Port: 80 Port forwarding rule directed external port 80 traffic to the internal web server IP.	Interface: WAN / Dest. Port: 443 The port forwarding rule has been updated to direct external port 443 traffic to the internal Reverse Proxy IP.

Recommendations

RECOMMENDATIONS

This section outlines recommendations for continued monitoring and next steps for future remediations to further strengthen MegaQuagga's security posture.

Recommendation #1: Automate OWASP Core Rule Set (CRS) Updates

The current ModSecurity WAF relies on a static version of the OWASP Core Rule Set. To ensure MegaQuagga remains protected against newly discovered vulnerabilities and evolving attack vectors, a cron job or automated script should be implemented to regularly check for and download the latest CRS updates. This ensures the WAF's signature database never becomes obsolete.

Recommendation #2: Implement Automated SIEM Alerting Thresholds

While Graylog is currently collecting logs, the security team must manually review them to find incidents. We should configure active alerting rules within Graylog to trigger immediate notifications (via email or Slack) when high-severity events occur—such as repeated 403 Forbidden errors from the same IP address or SQL injection patterns—allowing for a faster incident response time.

Recommendation #3: Replace Self-Signed Certificates with a Trusted CA

The current HTTPS implementation uses a self-signed certificate, which triggers browser security warnings and creates trust issues for users. We should replace this with a valid certificate issued by a trusted Certificate Authority (CA) like Let's Encrypt. This will eliminate browser warnings, improve user trust, and prevent "man-in-the-middle" attackers from easily spoofing the site.



Conclusion

CONCLUSION

We successfully hardened the MegaQuagga infrastructure by implementing a ModSecurity Web Application Firewall (WAF) in active prevention mode and deploying an encrypted HTTPS reverse proxy to secure network traffic. These critical remediations directly addressed major security gaps, specifically eliminating the risk of cleartext data interception and blocking common web application attacks such as SQL injection and Cross-Site Scripting (XSS). Moving forward, we recommend further strengthening the security posture by automating OWASP rule updates, configuring proactive SIEM alerting thresholds, and replacing self-signed certificates with those from a trusted Certificate Authority.